DA3408 Assignment 5 — SwiftParcel ETA serving (DA24B029)

A — REST service and capacity

The FastAPI service loads the model once at startup. Pydantic rejects out-of-range values, batches outside 1–500 rows and infeasible trips (distance over 150 km with more than 20 stops); these get a 422 response. Every response carries a request ID, and `/metrics` exposes Prometheus counters and latency histograms. All 64 tests pass.

I ran everything on an Ubuntu VM with 6 vCPUs (AMD Ryzen 7 250), 7.8 GiB RAM and no GPU. The load-test client ran on the same machine as the server.

Baseline (v1, one worker, batch size 1):

Concurrency	Req/s	p50 ms	p95 ms	p99 ms	Errors
1	156.5	6.1	9.6	13.7	0
2	187.2	10.0	15.7	30.5	0
4	121.0	30.1	60.4	74.8	0
8	75.2	98.2	193.8	229.2	0
16	70.5	219.1	363.0	428.2	0

Throughput peaks at two clients. After that, extra clients only add waiting time, and throughput drops because the client and server fight over the same CPUs. The best rate that keeps **p95 under 80 ms is 187.2 req/s at concurrency 2**. As a sanity check with Little's Law, two clients at 187.2 req/s works out to about 10.7 ms per request, which matches the measured p50 of 10.0 ms. These runs are short and noisy: a repeat run peaked at 155 req/s.

Batching (concurrency 1): a 1-row request handled 151 rows/s. A 100-row request took only about 2 ms longer (p50 8.3 ms vs 6.1 ms) and handled 11,450 rows/s, about **76 times more**. The per-request overhead (HTTP, parsing, validation) is paid once per batch instead of once per row. Each row still waits for its whole batch to finish, though.

Optimization, two Uvicorn workers: one Python process mostly uses one core, so I added a second worker process.

Workers	c=1	c=2	c=4	c=8	c=16
1 (req/s; p95 ms)	156.5; 9.6	187.2; 15.7	121.0; 60.4	75.2; 193.8	70.5; 363.0
2 (req/s; p95 ms)	138.5; 11.4	192.0; 17.4	**239.2; 34.3**	185.5; 116.0	170.8; 168.3

Capacity under the 80 ms target went from 187.2 to **239.2 req/s, about 1.28×**. That isn't double, because the load generator shares the same CPUs, but it's a clear improvement, so I'd keep two workers.

Averages hide the slow requests, which is why I report p95 and p99: the tail is what users actually notice, and it's what SLOs are written against.

B — Registry, promotion gate and shadow

Both models are registered in MLflow (SQLite registry) as `eta-model`, each with a signature, an input example and pinned dependencies. **Version 1 (v1) is `prod`, and version 2 (v2) is `candidate`.**

The gate compares the models on 2,000 fresh rows. It promotes only if all of these hold:

- overall MAE improves enough;
- candidate p95 latency is under 10 ms;
- neither the rush-hour slice nor the rain slice gets more than 5% worse.

`decide()` is covered by 105 passing tests.

Metric (fresh data)	prod v1	candidate v2
Overall MAE (min)	11.96	6.10
Rush-hour MAE (min)	19.68	8.58
Rain MAE (min)	11.62	6.29
Inference p95 (ms)	7.40	7.27

v2 cuts the error almost in half (**48.9% better MAE**) and improves both slices, so the **default gate promotes it**. With the strict setting `min_rel_gain=0.60`, it fails the gain check and is not promoted. Neither run moves the aliases.

Shadow test: I served both versions with `mlflow models serve` and sent the same 200 requests to each. They differed by **9.48 minutes on average**, and **35% of predictions differed by more than 10 minutes**. The p95 latency was 17.5 ms for prod and 16.6 ms for the candidate. The models clearly disagree a lot, but the labelled evaluation shows that v2's answers are the better ones.

Recommendation: move v2 to a small, monitored canary. Changing the alias alone does nothing to a server that's already running, because it keeps the model it loaded at startup. To switch without dropping requests, I would use either:

- **blue-green:** start v2, wait until it's ready, switch traffic over, then drain the old server;
- **rolling update:** replace servers one at a time, keeping old ones until the new ones are ready.

C — Sticky Ray Serve canary

Each customer ID is hashed with SHA-256 into a fixed bucket, and the canary weight just sets the cut-off. So a customer always gets the same model, even after a redeployment, and raising the weight only moves more customers onto v2, never back. Python's built-in `hash()` would not work here, because it changes every time a process starts. The `X-Beta: 1` header forces v2 for testers.

Live check	Result
Canary share at weight 0.2	213/1000 = 21.3% (target 16–24%)
Same customers, repeat calls	200/200 kept their model
After redeployment	200/200 kept their model
Weight raised to 0.5	0 of 213 canary customers went back to v1
`X-Beta: 1` header	Forced v2; normal routing restored afterwards
Batch predictions	Match v1 and v2 exactly

The candidate autoscales from 1 to 3 replicas, aiming for about 2 in-flight requests per replica. It waits 5 seconds before scaling up, to ignore short spikes, and 30 seconds before scaling down, to avoid flapping. All the replicas together reserve at most 1.5 of Ray's 2 CPUs.

D — Smoke test and canary abort rules

`smoke_eta.py` runs seven checks against a live server and exits 0 (pass) or 1 (fail):

1. health and model version;
2. a valid prediction;
3. batch consistency;
4. a longer distance never gives a shorter ETA;
5. heavier traffic never gives a shorter ETA;
6. invalid input returns 422;
7. p95 latency.

Scenario	Expected / actual exit	p95 ms
v1 release	0 / 0	14.84
Wrong version expected	1 / 1	—
Impossible latency limit	1 / 1	9.89
Server not running	1 / 1	—
v2 release	0 / 0	13.31

When I would abort a 10% canary of v2:

Metric	Abort if	Window
Errors (5xx and timeouts)	> 1% of requests (and at least 5 failures)	5 min, ≥ 500 requests
p95 latency	> 80 ms	2 consecutive 1-min windows
Overall MAE	> 1.05 × v1 MAE on the same deliveries	30 min of new labels, ≥ 200 pairs
Rush-hour or rain MAE	> 1.05 × v1 on that slice	60 min of new labels, ≥ 100 pairs

If any of these trips, I would stop the rollout, send new traffic back to v1, and let in-flight requests finish. If there isn't enough data yet, the rollout waits. These are proposed rules; I have not built automatic monitoring for them.

What I did not test

- Installing on a fresh machine.
- Ray scaling all the way to 3 replicas, or the exact scale-up/down timing.
- Recovery after a replica crashes.
- A real zero-downtime blue-green or rolling switch.
- Docker/Kubernetes serving.
- Live canary monitoring with automatic rollback.
- Load tests with the client on a separate machine, or repeated many times.

Evidence and sources

Test outputs:

- Part A: `loadtest_results.json`, `batch1_results.json`, `batch100_results.json` and `two_workers_results.json`.
- Part B: `registry_output.txt`, `gate_output.txt` and `shadow_output.txt`.
- Part C: `verify_output.txt`.
- Part D: `verify_output.txt`.
- Screenshots: in `screenshots/`.

Sources:

- The assignment PDF and starter files.
- The Lecture 10 notebooks on REST, MLflow, Ray Serve and Minikube serving, from `sudarsun/DA3408`.

AI assistance is described in `AI_DISCLOSURE.md`. I ran every experiment, and every number here comes from my own runs on 8 November 2026.